GDNet: Naming Security Products from the First Five Packets of a Flow

Author: lupingQAQ, independent researcher

Abstract

Networks run more security agents than any inventory lists, and during an incident that gap costs time. I built a small classifier that reads the first five packets of a flow and masks the addresses. It names the security product behind the flow, or says it does not know. Four 2.86M-parameter arms vote. On a 503-flow test split the ensemble reaches 0.8787 product accuracy and 0.9245 tier accuracy, the gate AUROC is 0.9784, and all 18 unknown-product bins are rejected. Along the way I audited six public benchmarks. Five of them share about 80% of their test flows with train, byte for byte; Tor is the only clean one. That finding is why this paper reports deduplicated numbers and declines the usual leaderboard claim. Code and weights are Apache-2.0; inference needs a CUDA GPU.

1 Introduction

An incident responder often starts with a simple question about a host: which security agents are installed on it? The inventory says one thing, the traffic says another. Operations installs agents, the network team deploys appliances, and the central list falls behind both.

GDNet answers a narrow version of that question. Given the first five packets of a flow, it names the security product that produced them, or says it does not know. It also predicts a type and a tier, and it carries a secondary malware head that I do not use for alerting. One arm has 2.86M parameters; the release averages four arms and runs on an 8 GB GPU. The probe is a single 42 KB C file. The Python package handles offline evaluation and an HTTP service.

Two decisions shaped everything else. Bytes instead of tokens: a tokenizer would have added a vocabulary and a parsing step, and the probe would have grown with them. And a strict split: the pretraining pool leaves out validation and holdout flows, and every choice that can leak, from the threshold to the ensemble, is made on dev.

The benchmarks need a separate admission. Five of the six public splits used in the internal comparison contain about 80% exact train/test overlap. Tor is the only clean one. A number measured on a split like that says more about memory than about generalization, so this paper reports a deduplicated evaluation and makes no leaderboard claim.

2 Related Work

2.1 Byte-level traffic classification

ET-BERT tokenizes datagrams and pretrains a Transformer with masked language modeling. YaTC represents flows as images and pretrains a masked autoencoder. NetMamba applies a unidirectional Mamba backbone to packet sequences. MambaNetBurst removes tokenization and pretrains directly on bytes; its 2.7M-parameter model is the closest architectural relative of this work.

The backbone here follows the byte-level design of MambaNetBurst but uses Gated DeltaNet blocks. The released implementation is a trimmed GDN-only encoder. No upstream source is redistributed.

2.2 Foundation models

netFound learns packet-level representations from a large token corpus and evaluates frozen features on downstream tasks. It reports strong frozen-encoder results and explicit corrections for dataset shortcuts. The model is roughly two orders of magnitude larger than one GDNet arm. An internal comparison under a fixed five-packet budget was competitive, but it is not reproduced here. Input budget and split construction differ enough that cross-paper numbers are hard to compare.

2.3 Open-set recognition

Prototypical networks classify a sample by its distance to class prototypes. I use three sub-prototypes per class to represent intra-class variation. At inference, the distance is one minus the maximum cosine similarity to any prototype. A threshold calibrated on dev decides whether to trust the closed-set product head. The method needs no unknown-class training data.

2.4 Dataset leakage

Trustee and Pcap-Encoder show that network ML systems often exploit dataset shortcuts, including packet-level splits and timestamp correlations. netFound reports corrected splits for several public datasets. The audit in Section 5 extends this line with exact byte-level duplicate counts for six widely used traffic benchmarks. It is independent of any model.

3 Method

3.1 Input

The probe emits a flow vector after the fifth packet. Waiting for a connection to close would delay every verdict, so the input stops there. Five packets also happen to be enough. They cover the handshake and the first application message, which is where a security product leaves most of its mark.

Each packet contributes 320 bytes: 80 from the IPv4 and transport headers, 240 from the payload. Five packets give 1,600 bytes. Bytes 12:20 hold the IPv4 source and destination addresses, and I zero them. An address is a property of the site, not of the product; a model that learns addresses will not transfer.

Parsing is dpkt's job. It reads Ethernet, VLAN, and raw IPv4 out of pcap and pcapng files. I did not use libpcap, because the dpkt wheel installs without a system dependency and parsing is not on the hot path.

Property	Value
Flow length	1600 bytes
Packets per flow	5
Bytes per packet	320
Header bytes per packet	80
Payload bytes per packet	240
Address handling	IPv4 source and destination zeroed
Direction	Unidirectional

3.2 Backbone

One arm is four GDN blocks at d_model=256 and d_state=16, with a 256-dimensional byte embedding. The CLS token is appended after the last byte; the final CLS state is the feature. The state dict has 103 entries. Fifty-

two belong to the GDN blocks; the rest are the embedding, the heads, and the training-only domain head. I load with strict=False, so the domain head can stay in the checkpoint without being instantiated at inference.

```
bytes (1600)
  -> ByteEmbed (256)
  -> [bytes, CLS] + positional embedding
  -> 4 x GDN (d_model=256, d_state=16)
  -> CLS feature (256)
```

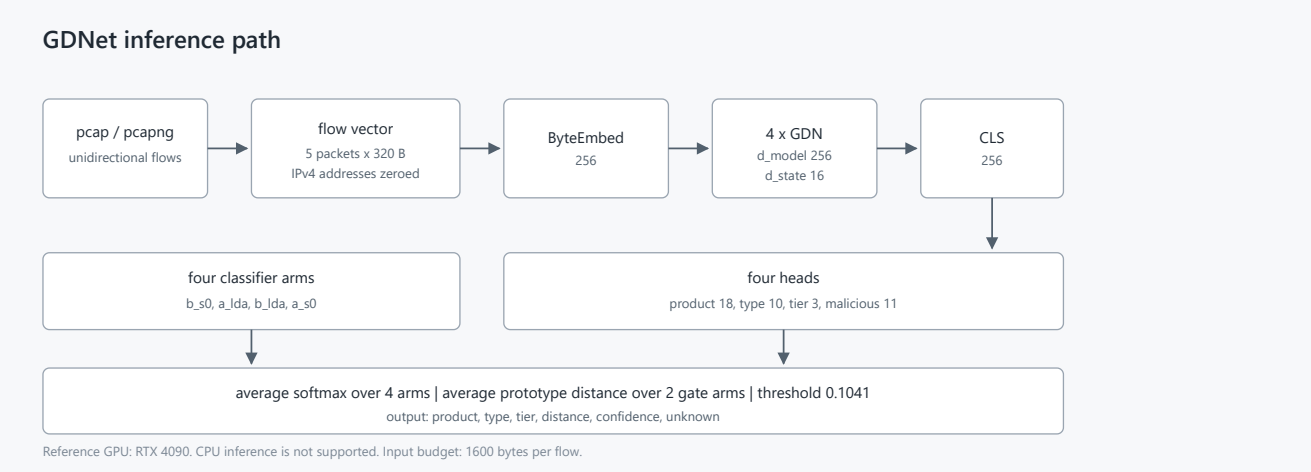


Figure 1. Inference path. The four-arm ensemble and two-arm gate are described in Section 3.5.

3.3 Task heads

Four heads sit on the CLS state, and only the product head drives the main metric. It is a small MLP with one hidden layer of 512 units. The tier head is a single linear layer; it exists because a coarse label stays useful when two products are confused. The type head and the malicious head reuse the same hidden width. The malicious head is trained on public malware traffic, but I do not use it in the main results and would not use it for alerting without a separate calibration. All four heads together hold about 0.4M parameters, so nearly everything else in the model is the encoder.

Head	Outputs	Purpose
Product	18	Security product or service
Type	10	WAF, behavior management, bastion host, NDR, zero trust/VPN, EDR/HIDS, antivirus, browser, file transfer, remote administration
Tier	3	Security appliance, security software, productivity software
Malicious	11	Benign plus ten malware families

3.4 Open-set decision

Rejection works by distance. For each product class I run KMeans with three clusters over the training embeddings, which gives 54 rows. At inference I normalize the feature, take the maximum cosine similarity over the rows, and subtract from one. The gate averages two arms. The threshold is 0.1041, the 68th percentile of dev distances. It is not a hand-picked value; it comes from the dev quantile. A new site should recalibrate it.

```
d = 1 - max cosine(feature, prototype)
```

3.5 Ensemble

Four arms made the cut. I dropped the two hard-negative arms because their gate distance moved when I changed the threshold, even though their AUROC was higher. The four remaining arms differ in pretraining data and loss weights. At inference I average four softmax vectors and two distance vectors. The ensemble has 11.44M parameters and a 46 MB footprint.

3.6 Training

Training went in three stages. Pretraining used 33,184 training-only flows. The pool excludes validation and holdout flows; the build script checks the intersection and fails if it is non-zero. SFT ran for 60 epochs, RL for 300 episodes. I trained on four RTX 4090s.

The training code is not in v0.1.0. It assumes the lab data layout, and I would rather publish a tested inference path than an untested training path. The checkpoints, prototypes, threshold, and evaluation script are enough to reproduce inference.

4 Evaluation

4.1 Data and splits

The labeled set has 40,283 flows across 18 classes, all from lab captures. The validation set has 1,006 flows from 14 held-out sessions. I split it into dev and test at flow level, 503 each. Threshold, checkpoint, and ensemble choices use dev. Test is used once.

The 18 classes are `windows_defender`, `clamav_freshclam_win`, `clamav_freshclam`, `velociraptor`, `velociraptor_win`, `osquery`, `wazuh_agent`, `wireguard`, `openvpn`, `stunnel`, `squid_proxy`, `ntopng`, `guacamole_mgmt`, `waf_modsecurity`, `suricata_update`, `web_http`, `ssh`, and `ftp`. Captures came from lab hosts and virtual machines. Each product was exercised through its normal update or management path, not through synthetic payloads.

The split is at flow level, not session level. Each class has one held-out session. The numbers describe those sessions, not a new organization.

4.2 Metrics

- Product accuracy counts exact product labels.
- Tier accuracy counts exact tier labels.
- Macro-F1 averages over classes with at least one test flow.
- Gate AUROC compares known flows with unknown and novel flows.
- Unknown rejection is the share of unknown bins above the threshold.
- Malicious macro-F1 uses the deduplicated USTC-TFC2016 test split.

4.3 Results

Metric	Test (503 flows)	All validation (1006 flows)
Product accuracy	0.8787	0.8817
Tier accuracy	0.9245	0.9334
Macro-F1, supported classes	0.7621	0.8240
Gate AUROC	0.9784	0.9820 (dev)

The table is the summary; two entries deserve comment. Macro-F1 on the test half leans on classes with one or two flows — `wireguard` has exactly one — so I quote the all-validation value, 0.8240, instead. Everything else reads as measured.

The unknown bins matter more to me than the accuracy. All 18 rejected at the fixed threshold, stable from q=0.64 to q=0.75. The malicious head, on the deduplicated USTC test set of 1,242 flows, reaches 0.9702 macro-F1; binary F1 stays above 0.994.

4.4 Acceptance and enrollment

The acceptance set has 264 product bins and 18 unknown bins. The raw ensemble gets 8 of 10 products right. A timing rule that uses inter-arrival times raises this to 9, but the rule is not in the release. I did not want to ship a second model without its own calibration. Unknown bins are rejected at 100% in both settings.

`stunnel` is the failure: 0 of 30 bins. `wireguard` has two bins; I treat that as noise.

The HTTP service returns 6/6 known flows on FLW1 records. Enrollment reaches 8/8 after three sub-prototypes for `smb`.

4.5 Throughput and memory

On one RTX 4090 a single arm classifies about 1,017 flows per second and the four-arm ensemble about 253. Batch-1 latency is 10.7 and 43.1 milliseconds. The ensemble peaks near 3.4 GB at batch 128, so an 8 GB card is comfortable. The probe captures faster than the ensemble classifies, which means real deployments sample, shard, or classify on demand.

5 Public Split Audit

5.1 Method

I hash every flow in the distributed train, validation, and test directories. Two hashes are used: MD5 of the PNG file bytes, and MD5 of the decoded grayscale pixels. The two hashes agree. Duplicates are exact byte-level matches, not near-duplicates. The input unit is 1,600 bytes, matching the model input.

The audit covers six benchmarks used in the internal comparison: ISCXVPN2016, ISCXTor2016, USTC-TFC2016, CICIoT2022, CrossPlatform-Android, and CrossPlatform-iOS. The scripts ship with the code.

5.2 Results

Benchmark	Train	Test	Train ∩ test	Share of test	Clean test
ISCXVPN2016	13,281	1,384	1,106	79.9%	278
ISCXTor2016	11,655	1,458	0	0%	1,458
USTC-TFC2016	37,323	6,662	5,605	84.1%	1,057
CICIoT2022	18,734	1,951	1,571	80.5%	380
CrossPlatform-Android	44,700	4,656	3,733	80.2%	923
CrossPlatform-iOS	40,379	4,205	3,396	80.8%	809

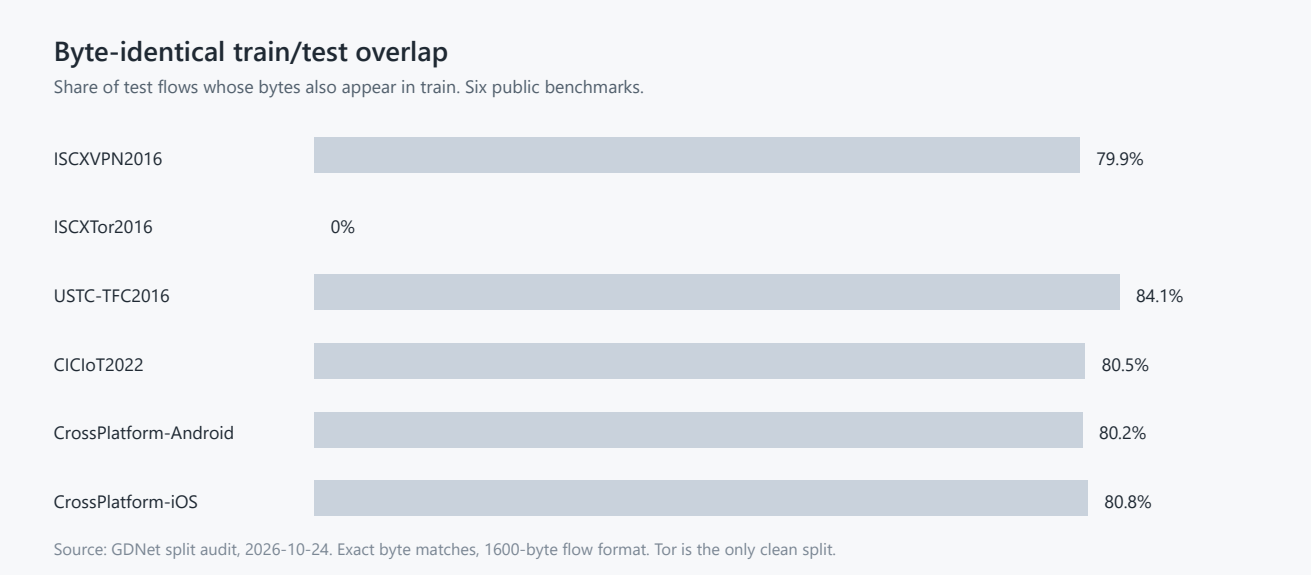


Figure 2. Exact byte-level train/test overlap in six public benchmarks. Tor is the only clean split.

Label conflicts across splits are zero. Every duplicate stays within its class. The likely cause is independent sampling from the same flow pool without cross-split deduplication. That is a property of the data distribution, not of a single paper.

5.3 Impact

Eighty percent overlap turns a test set into a memory quiz. A model can score 0.99 by recognizing flows it saw in training, and five of these six benchmarks reward exactly that. Comparisons inside one split still mean something, since both sides get the same head start. As evidence of generalization they mean nothing.

Tor is the one benchmark where nothing is duplicated. It is also where the internal report puts its strongest cross-model evidence. For the other five I report the original split and the deduplicated subset side by side.

5.4 Deduplicated evaluation

VPN, USTC, and CIoT were the three splits worth rerunning clean. Their clean test sets hold 278, 1,057, and 380 flows. Sets that small wobble, so I read the result as a drop, not a ranking. Tor needed no rerun.

Benchmark	Clean test	Accuracy	Macro-F1	Full-set macro-F1	Change
ISCXVPN2016	278	0.9376	0.9418	0.9861	-4.4 pp
USTC-TFC2016	1,057	0.9981	0.9671	0.9949	-2.8 pp
CICIoT2022	380	0.9868	0.9823	0.9965	-1.4 pp

Losing the duplicates costs 1.4 to 4.4 macro-F1 points. The internal report re-scores the external frozen-encoder baseline on these same clean subsets, and the lead over it widens to ten or seventeen points. That comparison stays out of this paper; the checkpoints and data behind it are not mine to redistribute.

5.5 What did not work

Attempt	Result	Decision
Auxiliary pretraining (field, order, reconstruction)	Probe +9.35 points; downstream E1 below E0+LDA	Not used
Timing rescue	Acceptance 8 to 9; unstable at nearby thresholds	Not released
Distillation	0.8231 validation accuracy; 5.6 points below the ensemble	Experimental only
`stunnel` threshold tuning	0/30 at every threshold	Needs data or enrollment

None of these shipped. A bigger model does not fix the split, and a part that only helps dev is not worth carrying.

6 Limitations

The biggest limitation is the split, not the model. Each class has one held-out session, so the test numbers describe those sessions, not a new site. A second session per class would be more useful than another point of accuracy.

`stunnel` is not fixed. I tried enrollment, and the 8/8 result on `smb` shows the mechanism works, but `stunnel` needs its own data.

The timing rule used in the 9/10 acceptance result is not in v0.1.0. The released ensemble reproduces the 8/10 raw result.

CPU inference is not supported. The GDN backbone uses Triton kernels and needs a CUDA GPU. A CPU port would be a separate implementation, not a build flag.

The model was trained on lab traffic. Performance on production networks with different software versions, proxies, or NAT configurations is unknown.

The label set is closed at 18 products plus an unknown class. Products outside the set are rejected or misclassified.

Training code is not in v0.1.0. The checkpoints, prototypes, threshold, and evaluation script are enough for inference and evaluation, not for retraining. The training scripts depend on the lab data layout and need cleanup before they are useful.

The public benchmark comparison is same-protocol. I do not claim state-of-the-art accuracy.

7 Release

Small on purpose. One repository, one model repository, one tag, no installer. The weights folder weighs 94.6 MB, four arms at 46 MB of it, and every file has a checksum next to it.

The CLI writes one row per flow and JSON when asked. Evaluation is a folder of 1600-byte bins plus a CSV of labels. The split stays private; I am not publishing captures.

One RTX 4090, Python 3.12, PyTorch 2.14 — that is the whole tested surface. No CPU path, on purpose: the GDN kernel is Triton-only, and a silent fallback would lie to the user. Code at github.com/lupingQAQ/gdnet, weights at huggingface.co/luping123/gdnet, tag v0.1.0. Apache-2.0 for code and weights, CC-BY-4.0 for this text.

8 Use and misuse

A tool that names defenses serves the owner first and an attacker just as well, so the rule is the boring one: run it where you have permission, and keep captures off public demos.

The outputs are scores, not verdicts. A borderline label or an unknown goes to a human before anything happens. The malicious head is the part most open to misuse; I trained it on public malware traffic, kept it out of the main results, and it does not belong in an automated response path.

The input has no addresses and no payload semantics. The model sees byte patterns, so it cannot name a user or reconstruct content.

Appendix A: Label Set

Product	Type	Tier
windows_defender	Antivirus	Security software
clamav_freshclam_win	Antivirus	Security software
clamav_freshclam	Antivirus	Security software
velociraptor	EDR / HIDS	Security software
velociraptor_win	EDR / HIDS	Security software
osquery	EDR / HIDS	Security software
wazuh_agent	EDR / HIDS	Security software
wireguard	Zero trust / VPN	Security appliance
openvpn	Zero trust / VPN	Security appliance
stunnel	Zero trust / VPN	Security appliance
squid_proxy	Internet behavior management	Security appliance
ntopng	NDR	Security appliance
guacamole_mgmt	Bastion host	Security appliance
waf_modsecurity	WAF	Security appliance
suricata_update	NDR	Security appliance
web_http	Browser	Productivity software
ssh	Remote administration	Productivity software
ftp	File transfer	Productivity software

References

1. MambaNetBurst: direct byte-level network traffic classification without tokenization or pretraining. arXiv:2605.11034, 2026.
2. NetMamba, efficient traffic classification with a pre-trained unidirectional Mamba. arXiv:2405.11886 (2024).
3. ET-BERT: a contextualized datagram representation with pre-training transformers for encrypted traffic classification. WWW 2022; also arXiv:2110.02633.
4. YaTC, yet another traffic classifier: a masked autoencoder over flow images. arXiv:2305.19077.
5. netFound, a foundation model for network security. arXiv:2310.17025, first posted 2023, revised 2026.
6. Gated Delta Networks: improving Mamba2 with the delta rule. arXiv:2412.06464 (2024).
7. Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. NeurIPS 2017, arXiv:1703.05175.
8. The Sweet Danger of Sugar (Pcap-Encoder). ACM SIGCOMM 2025, arXiv:2507.16438.
9. The Emperor Has No Clothes: AI/ML for network security. ACM CCS 2022.
10. NetBench, a benchmark for network traffic classification. ACM FMSys 2024.
11. flash-linear-attention, source repository. github.com/fla-org/flash-linear-attention.
12. PyTorch: an imperative style, high-performance deep learning library. NeurIPS 2019.

Nothing in the repository is a capture. The weights saw lab traffic and public benchmark training splits, and the checksums let anyone verify which files they are loading.